

Editorial — Esquenta Maratona de Programação 2026

Problema A — Placar do Esquenta

Solução

Ideia

O problema pede o **máximo de uma tabela segundo uma ordem de três chaves**: primeiro o número de problemas resolvidos (quanto maior, melhor), depois a penalidade (quanto menor, melhor) e, por último, o número de inscrição (quanto menor, melhor). Como as três chaves juntas nunca deixam duas equipes indistinguíveis — os números de inscrição são todos diferentes —, a resposta é **única**.

A tentação é guardar as n equipes num vetor e ordenar. Funciona, mas é trabalho a mais: ordenar custa $O(n \ln n)$, exige escrever um comparador de três chaves e obriga a guardar a tabela toda na memória. Achar o máximo de uma sequência não precisa de nada disso — basta **uma varredura**, lembrando apenas de qual é a melhor equipe vista até agora.

A observação-chave está no desempate por número de inscrição. Como a leitura acontece na ordem $i=1, 2, \dots, n$, quem chega **primeiro** já tem o menor número. Então basta trocar a campeã provisória apenas quando a equipe nova é **estritamente** melhor:

- $p_i > p_{\text{melhor}}$; ou
- $p_i = p_{\text{melhor}}$ e $c_i < c_{\text{melhor}}$.

Empate total (mesmo p e mesma c) não satisfaz nenhuma das duas condições, então a campeã provisória — que entrou antes e portanto tem número menor — permanece. O desempate por número sai de graça, sem nenhum if extra: ele é consequência de usar comparação estrita, e não $\geq$. Trocar qualquer um dos dois $>$ por $\geq$ faz o programa devolver a **última** equipe empatada em vez da primeira, e é exatamente esse o erro que os testes com empate caçam.

Para começar a varredura sem tratar a primeira equipe como caso especial, inicializamos a melhor com $p_{\text{melhor}} = -1$: como todo $p_i \geq 0$, a equipe 1 obrigatoriamente vence essa comparação e assume o posto.

Um cuidado de tipo: c_i chega a 10^9 , que ainda cabe num inteiro de 32 bits

com sinal (máximo 2147483647). Como aqui só há **comparação** de penalidades — nunca soma nem produto — não existe risco de estouro. Ainda assim, guardar p , c e o índice em inteiros de 64 bits não custa nada e elimina a dúvida.

Passos

1. Ler n .
2. Inicializar $\text{melhor} = 1$, $\text{melhorP} = -1$ e $\text{melhorC} = 0$.
3. Para i de 1 até n :
 1. ler p_i e c_i ;
 2. se $p_i > \text{melhorP}$, ou se $p_i = \text{melhorP}$ e $c_i < \text{melhorC}$, então atualizar $\text{melhor} = i$, $\text{melhorP} = p_i$ e $\text{melhorC} = c_i$;
 3. caso contrário, não fazer nada (a campeã provisória continua).
4. Imprimir melhor .

Repare que a tabela nunca precisa ser armazenada: cada linha é consumida assim que é lida.

Complexidade

A varredura visita cada equipe exatamente uma vez e faz um número constante de comparações por equipe (no máximo duas), logo o **tempo** é $O(n)$. No pior caso, $n=10^5$ equipes: são 10^5 iterações, $2 \cdot 10^5$ inteiros lidos da entrada e no máximo $2 \cdot 10^5$ comparações — algo como 10^6 operações elementares contando a conversão dos dígitos, muito abaixo do que qualquer linguagem faz em um segundo. O custo real é dominado pela **leitura** da entrada, que tem até cerca de 1,4 MB; por isso vale usar leitura em bloco (`fread`, `DataInputStream`, `sys.stdin.buffer.read()`, `read_to_string`) em vez de leitura formatada linha a linha.

A **memória** é $O(1)$: guardamos apenas n , o contador do laço e os três valores da campeã provisória. Nenhum vetor de tamanho n é alocado — o que também torna o programa imune ao tamanho da tabela.

Problema B — Detector de Plágio

Solução

Ideia

A abordagem ingênua — para cada assinatura, varrer S inteiro — custa $O(k \cdot |S|)$ chamadas de busca: com $k=10^5$ e $|S|=10^6$ são 10^{11} posições

examinadas no pior caso. Não termina. E não adianta procurar uma assinatura de cada vez: o texto tem de ser lido **uma única vez**, com todas as assinaturas sendo procuradas ao mesmo tempo.

Quem faz isso é o **autômato de Aho-Corasick**. Ele é a generalização do KMP para vários padrões:

1. **Trie** de todas as assinaturas. Cada nó representa um prefixo de alguma assinatura; a raiz é a cadeia vazia. Como $\sum |P_i| \leq 2 \cdot 10^5$, a trie tem no máximo $2 \cdot 10^5 + 1$ nós.
2. **Link de falha**: $fail[u]$ é o nó correspondente ao **maior sufixo próprio** da cadeia de u que também é um nó da trie. É o mesmo papel da função de falha do KMP — quando a leitura trava, ela diz para onde recuar sem reler o texto.
3. **Autômato completo**: em vez de seguir links de falha durante a leitura, pré-calculamos a transição de **todo** nó por **toda** letra. Se o nó u não tem filho pela letra c , definimos $go[u][c] = go[fail[u]][c]$. Isso é calculado numa BFS por níveis: quando processamos u , o nó $fail[u]$ (que é mais raso) já está pronto. Com o autômato completo, ler S é uma passada de $|S|$ transições, cada uma $O(1)$ — sem laço interno.

A observação que fecha o problema é a da **contagem**. Depois de consumir os primeiros i caracteres de S , o autômato está no nó u = maior sufixo do que já foi lido que é prefixo de alguma assinatura. As assinaturas que terminam na posição i são exatamente as que aparecem como **sufixo** da cadeia de u — ou seja, as marcadas em u , em $fail[u]$, em $fail[fail[u]]$, e assim por diante. Percorrer essa cadeia a cada caractere seria $O(|S| \cdot \text{profundidade})$ e volta a estourar (o caso $S = \text{aaa} \dots \text{a}$ com as assinaturas $a, aa, aaa, \dots$ é exatamente esse desastre).

A saída é **propagar a contagem uma vez só**, antes da leitura: $cnt[u] \leftarrow cnt[u] + cnt[fail[u]]$ na **ordem da BFS**. Como $fail[u]$ está sempre num nível anterior, quando chegamos em u o valor de $cnt[fail[u]]$ já acumulou toda a cadeia. Depois disso, $cnt[u]$ é exatamente *quantas assinaturas terminam aqui*, e a leitura de S vira

$$\text{resposta} = \sum_{i=1}^{|S|} cnt[u_i],$$

com u_i o nó do autômato após ler o i -ésimo caractere.

Dois detalhes que valem vereditos:

- **Nunca liste as ocorrências**, só conte. A resposta chega a 10^{11} ; enumerar par a par (assinatura, posição) é impossível por definição.

- **A resposta não cabe em 32 bits.** Com 10^5 assinaturas iguais a aa e S com 10^6 letras a , o total é $10^5 \times 999999 \approx 10^{11}$ — o acumulador tem de ser de 64 bits. (As contagens por nó cabem em 32 bits: nenhuma passa de k .)

Uma última decisão que faz diferença na prática: guardar o autômato em **vetores planos** (go com $26 \cdot n$ inteiros, $fail$ e cnt com n) em vez de um nó-objeto com dicionário por letra. É o que mantém a constante baixa e a memória previsível — e permite a mesma implementação, letra por letra, em todas as linguagens.

Passos

1. Ler toda a entrada de uma vez (o texto tem até 1 MB; leia **bytes**, não caracteres Unicode).
2. Somar os comprimentos das assinaturas em $T = \sum |P_i|$ e alocar go com $26(T+1)$ inteiros, $fail$ com $T+1$, cnt com $T+1$ inteiros de 64 bits e a fila q com $T+1$.
3. Montar a trie: para cada assinatura, caminhar da raiz criando os nós que faltam; ao terminar, fazer $cnt[u] += 1$. A raiz é o nó 0, então $go[\dots] == 0$ já significa “sem filho”.
4. BFS: empilhar os filhos da raiz com $fail = 0$; ao tirar u da fila, para cada letra c : se o filho v existe, $fail[v] = go[fail[u]][c]$ e v entra na fila; se não existe, $go[u][c] = go[fail[u]][c]$.
5. Percorrer a fila **na ordem em que foi preenchida** fazendo $cnt[u] \leftarrow cnt[u] + cnt[fail[u]]$.
6. Ler S da esquerda para a direita: $u \leftarrow go[u][s_i]$ e somar $cnt[u]$ num acumulador de 64 bits.
7. Imprimir o acumulador.

Tudo é iterativo: a trie é um laço, a BFS usa uma fila em vetor e a leitura é uma passada. Não há recursão em lugar nenhum, o que também evita estouro de pilha com assinaturas longas.

Complexidade

Seja $T = \sum |P_i| \leq 2 \cdot 10^5$, $\Sigma = 26$ o alfabeto e $|S| \leq 10^6$.

- **Tempo:** $O(T)$ para montar a trie, $O(T \cdot \Sigma)$ para a BFS que completa o autômato, $O(T)$ para propagar as contagens e $O(|S|)$ para ler o texto — total $O(T \cdot \Sigma + |S|)$. No pior caso são $2 \cdot 10^5 \cdot 26 = 5,2 \times 10^6$ transições calculadas na BFS mais 10^6 transições na leitura, algo como $6,2 \times 10^6$ operações elementares. Sobra tempo.
- **Memória:** dominada pela tabela de transições, $26 \cdot (T+1)$ inteiros de

32 bits $\approx 5,2 \times 10^6$ inteiros ≈ 21 MB, mais $O(T)$ para fail, cnt e a fila e $O(III)$ para o texto. Total abaixo de 30 MB — cabe folgado no limite de memória.

Para comparação, a busca ingênua com uma varredura por assinatura faz até $k \cdot S \cdot I = 10^5 \cdot 10^6 = 10^{11}$ comparações. Medido na máquina de desenvolvimento (sols/slow/tle.cpp, uma busca por assinatura com `string::find`): no teste de resposta máxima ela passa de **300 s** sem terminar, enquanto a referência em C fecha o mesmo teste em **0,03 s** — quatro ordens de grandeza de diferença.

Problema C — Rota até o Sabiazinho

Solução

Ideia

O mapa é um **grafo não dirigido com pesos positivos**: cruzamentos são vértices, ruas são arestas, o tempo de cada rua é o peso. O que se pede é o caminho de **peso mínimo** do vértice 1 ao vértice n — e o segundo exemplo já mostra que *não* é o caminho com menos ruas: a rua direta de 100 minutos perde para o desvio de três ruas que soma 12. Logo, busca em largura está fora; o que resolve é **Dijkstra**.

A observação que sustenta o Dijkstra é esta: se todos os pesos são ≥ 1 , então, entre os vértices ainda não fechados, aquele com a **menor** distância tentativa já está com a distância **definitiva** — qualquer outro caminho até ele passaria antes por um vértice de distância maior ou igual e ainda somaria pelo menos mais uma rua, nunca ficando menor. Então basta repetir: retire o vértice de menor distância tentativa, feche-o e relaxe as ruas que saem dele.

As alternativas óbvias não cabem nos limites: Floyd-Warshall custa $n^3 = 10^{15}$ operações e Bellman-Ford custa $n \cdot m = 2 \cdot 10^{10}$. Só o Dijkstra com **fila de prioridade** dá conta de $n = 10^5$ e $m = 2 \cdot 10^5$.

Três detalhes do enunciado se resolvem sozinhos, sem nenhum tratamento especial:

- **ruas repetidas**: o teste de relaxamento `nd < dist[to]` guarda automaticamente a melhor das duas; a pior nunca melhora ninguém;
- **laços** ($u = v$): relaxar v a partir de v dá `dist[v] + t > dist[v]`, porque $t \geq 1$;
- **desconexo**: se ao final `dist[n]` continua no infinito, nenhum

caminho existe e a resposta é -1 .

Duas decisões de implementação valem a pena, e são elas que fazem a mesma solução caber confortavelmente em todas as linguagens:

1. Lista de adjacência em formato CSR (três vetores planos). Em vez de um vetor de vetores (ou, pior, um mapa de hash), guarde as m ruas em três vetores eu , ev , ew , conte o grau de cada cruzamento, faça a **soma prefixa** desses graus em $start[1..n+1]$ e despeje as $2m$ pontas em $adjv/adjw$. As ruas do cruzamento v ficam contíguas nas posições $start[v] .. start[v+1]-1$. É uma ordenação por contagem: custa $O(n+m)$, faz **uma** alocação por vetor e percorre memória sequencialmente.

2. Fila de prioridade com remoção preguiçosa e chave empacotada. Não existe “diminuir a chave”: quando um relaxamento melhora $dist[to]$, simplesmente **insira uma nova entrada**; ao retirar uma entrada cuja distância seja maior que a $dist$ atual do vértice, **descarte-a**. Como toda inserção acontece junto com uma diminuição estrita de $dist[to]$, e cada aresta orientada consegue provocar no máximo uma diminuição, a fila recebe no máximo $2m+1$ entradas.

A chave inserida é **um único inteiro de 64 bits** — a distância deslocada 20 bits para a esquerda, com o número do vértice ocupando os bits baixos:

$$chave = dist \times 2^{20} + v$$

Isso funciona porque $v \leq 10^5 < 2^{20} = 1048576$ (o vértice cabe nos 20 bits baixos) e $dist \leq 10^{11} < 2^{37}$, então a chave fica abaixo de 2^{57} e sobra folga no `long long`. Comparar chaves é comparar primeiro a distância e, no empate, o número do vértice — exatamente o que um par ordenado faria, mas com uma comparação de inteiro só. A vantagem prática é que o heap vira um **vetor de inteiros** em qualquer linguagem: `priority_queue<long long>` em C++, `heapq` sobre uma lista de inteiros em Python, `BinaryHeap<Reverse<i64>>` em Rust e um heap binário escrito à mão sobre `long[]` em C, Java e Kotlin. Em Java e Kotlin isso é o que evita a `PriorityQueue`, que empacotaria cada entrada num objeto e pagaria alocação e indireção a cada inserção.

Por fim, a armadilha aritmética: a resposta pode chegar a $(n-1) \cdot 10^6 \approx 10^{11}$, muito acima do que cabe em 32 bits. Distâncias, o acumulador do relaxamento e a chave empacotada têm de ser de **64 bits** (`long long`, `long`, `i64`).

Passos

1. Ler n , m e as m ruas nos vetores eu , ev , ew (com leitura rápida: são até $6 \cdot 10^5$ números).

2. Contar em $start[v]$ quantas ruas tocam cada cruzamento v (cada rua conta nas duas pontas).
3. Transformar $start$ em soma prefixa, de modo que $start[v]$ seja a primeira posição do bloco de v , e copiar $start$ para um vetor auxiliar $fill$.
4. Para cada rua (u, v, t) , gravar $adjv[fill[u]] = v$, $adjw[fill[u]] = t$, incrementar $fill[u]$, e fazer o simétrico na ponta v (a rua é de mão dupla).
5. Inicializar $dist[1] = 0$ e $dist[v] = \text{INFINITO}$ para os demais; inserir na fila a chave $(0 \ll 20) \dot{=} 1 \dot{}$.
6. Enquanto a fila não estiver vazia: retirar a menor chave, separar $d = \text{chave} \gg 20$ e $v = \text{chave} \& 1048575$; se $d > dist[v]$, a entrada está velha — descartar e continuar.
7. Senão, para cada posição i de $start[v]$ até $start[v+1]-1$: se $d + adjw[i] < dist[adjv[i]]$, atualizar essa distância e inserir a chave $(nova_dist \ll 20) \mid adjv[i]$.
8. Ao final, imprimir $dist[n]$, ou -1 se ela continuar no infinito.

Complexidade

Tempo: $O((n+m) \lg n)$. A montagem do CSR é $O(n+m)$. Cada aresta orientada é percorrida uma única vez (o vértice de origem só tem suas ruas varridas quando sai da fila com a distância definitiva), o que dá $2m = 4 \cdot 10^5$ relaxamentos. A fila recebe no máximo $2m+1 = 400001$ entradas, e cada inserção ou remoção sobe/desce um heap binário de altura $\lg_2(4 \cdot 10^5) \approx 19$: são cerca de $1,5 \times 10^7$ comparações de inteiro no pior caso ($400001 \times 19 \times 2$). Somando a leitura de $6 \cdot 10^5$ números, o total fica na casa de 2×10^7 operações simples — folgado para o limite de tempo, inclusive nas linguagens interpretadas, já que tudo se resume a comparar e trocar inteiros em vetores planos.

Memória: $O(n+m)$. São três vetores de m inteiros para as ruas lidas (2,4 MB), dois vetores de $2m$ inteiros para o CSR (3,2 MB), o vetor $dist$ com n inteiros de 64 bits (0,8 MB) e o heap com até $2m+1$ inteiros de 64 bits (3,2 MB): cerca de **10 MB** no total, bem abaixo do limite. Nada de estruturas encadeadas, objetos por aresta ou mapa de hash — todo o consumo é previsível e contíguo.

Problema D — Veredito do MOJ

Solução

Ideia

O enunciado descreve uma **cadeia de decisão com prioridade**: as três condições não são independentes, elas são testadas **em ordem**, e a primeira que vale decide sozinha o veredito. É exatamente assim que o juiz de verdade funciona — internamente cada veredito tem um número de gravidade (TLE acima de WA, que fica acima de AC) e vence o mais grave.

Traduzir isso para código é escrever um `if / else if / else`, nesta ordem e não em outra:

1. $t > L \rightarrow \text{Time Limit Exceeded}$;
2. senão, $r == 0 \rightarrow \text{Wrong Answer}$;
3. senão $\rightarrow \text{Accepted}$.

Os dois pontos em que dá para escorregar são de **comparação**, não de algoritmo:

- **o limite é inclusivo**: gastar exatamente $t=L$ **não** é estouro. Quem escreve $t \geq L$ erra todos os casos de igualdade — e é por isso que o segundo exemplo tem $t=L$.
- **a ordem importa**: uma submissão pode ter a resposta certa ($r=1$) e estourar o tempo. Quem testa r antes de t responde Accepted onde o juiz responde Time Limit Exceeded, como no terceiro exemplo.

Note também que não existe nenhuma combinação de entrada que não caia em exatamente um dos três casos: as condições são mutuamente exclusivas e cobrem tudo, porque o `else` final é irrestrito.

Passos

1. Ler os três inteiros t , L e r de uma única linha.
2. Se $t > L$, imprimir Time Limit Exceeded e encerrar.
3. Senão, se $r=0$, imprimir Wrong Answer e encerrar.
4. Senão, imprimir Accepted.

Um cuidado de tipo: t e L chegam a 10^9 , que ainda cabe em um inteiro de 32 bits com sinal (cujo máximo é 2147483647), então `int` basta. Como aqui só há **comparação** — nunca soma ou produto — não há risco de estouro. Ainda assim, usar 64 bits não custa nada e elimina a dúvida.

Complexidade

O programa lê três números e avalia no máximo duas comparações, sem laço nenhum: **tempo** $O(1)$ e **memória** $O(1)$, independentemente dos valores da entrada. O custo real é dominado pela leitura da entrada, que tem tamanho constante.

Problema E — Cartazes do Ginásio

Solução

Ideia

O que o problema pede é a **área da união de n retângulos** de lados paralelos aos eixos.

As três tentativas ingênuas morrem rápido:

- **somar as áreas** conta duas vezes toda parede coberta por dois cartazes;
- **inclusão-exclusão** exige percorrer os 2^n subconjuntos;
- **comprimir as coordenadas e varrer a grade** é correto — as $2n$ abscissas e as $2n$ ordenadas cortam o plano em no máximo $(2n-1) \times (2n-1)$ células, e cada célula está inteira dentro ou inteira fora da união — mas são até 10^{10} células para $n=5 \cdot 10^4$, sem contar o custo de decidir cada uma.

A saída é a **varredura (sweep line)**. Imagine uma reta vertical caminhando da esquerda para a direita. Ela só muda de “estado” nas abscissas onde um cartaz começa (x_1) ou termina (x_2): entre duas abscissas consecutivas o conjunto de cartazes cortados pela reta é o mesmo, logo o **comprimento coberto no eixo y é constante** naquela faixa vertical. Se a faixa tem largura w e comprimento coberto c , ela contribui exatamente $w \cdot c$ para a resposta.

Isso reduz o problema a manter, sob **inserções e remoções de intervalos** $\mathcal{I}$, o comprimento da **união dos intervalos ativos**. Cada cartaz vira dois eventos: em x_1 o intervalo $\mathcal{I}$ entra; em x_2 ele sai.

A estrutura clássica para isso é a **árvore de intervalos de Klee** — uma árvore de segmentos construída sobre os intervalos elementares da compressão de y . Se os valores distintos de y são $y^{(0)} < y^{(1)} < \dots < y^{(m-1)}$, as folhas da árvore são os $m-1$ intervalos elementares $\mathcal{I}$, e todo intervalo que aparece na entrada é uma união exata de folhas consecutivas. Cada nó

guarda dois campos:

- `cover` — **quantos** intervalos ativos cobrem o **trecho inteiro** daquele nó, contados só nos nós onde a decomposição canônica do intervalo parou;
- `len` — o comprimento coberto **dentro** do trecho daquele nó.

E `len` é recalculado por uma regra local de três casos:

- se `cover > 0`, o nó está todo coberto: `len` = largura total do trecho do nó;
- senão, se o nó é folha: `len` $\hat{=}$ 0;
- senão: `len` = `len` do filho esquerdo + `len` do filho direito.

O ponto sutil — e o motivo de esta árvore **não** ser uma árvore de segmentos comum — é que **não existe push-down**. Numa árvore com propagação preguiçosa a informação pendente precisa descer antes de qualquer consulta; aqui ela pode ficar parada no nó para sempre, porque:

1. a única consulta é `len` da **raiz**, e a regra de três casos já resolve a raiz sozinha (se a raiz tem `cover > 0`, o valor certo é a largura total, não importa o que os filhos digam);
2. as operações vêm **casadas**: todo +1 feito em x_1 tem exatamente o -1 correspondente em x_2 , aplicado **nos mesmos nós** (o intervalo i é o mesmo, então a decomposição canônica é a mesma). Logo `cover` nunca fica negativo e nunca “vaza” para outro nó.

Repare também que `len` de um nó **ignora** o `cover` dos ancestrais — ele responde “quanto deste trecho está coberto por intervalos que pararam **aqui ou abaixo**”. Isso é proposital: quem consome `len` de um nó é o pai, e o pai só usa esse valor quando o próprio `cover` dele é zero.

Como cada inserção/remoção mexe em $O(\lg n)$ nós e cada faixa vertical custa $O(1)$, o total é $O(n \lg n)$.

Dois armadilhas de implementação valem nota:

- **a resposta chega a 10^{18}** (um único cartaz de $10^9 \times 10^9$), o que exige inteiro de 64 bits no acumulador. Os valores intermediários são pequenos — `len` da raiz nunca passa de 10^9 e a largura da faixa também não —, mas o **produto** e a soma acumulada precisam de 64 bits. Em Python o estouro não aparece; em C, Java, Kotlin ou Rust, usar 32 bits dá resposta errada em quase todo teste grande;
- **encostar não é sobrepor**. Como os intervalos são tratados como semiabertos i e a faixa vertical também é semiaberta, dois cartazes que dividem uma borda são contados uma vez cada, sem interseção — que é exatamente o que o enunciado pede.

Passos

1. Ler n e os n retângulos em quatro vetores planos x_a, y_l, x_b, y_h .
2. **Comprimir as coordenadas y** : copiar os $2n$ valores y_1, y_2 para um vetor, ordenar, remover os repetidos (sobram m valores, $2 \leq m \leq 2n$) e trocar cada y_1 e y_2 pelo seu índice, achado por **busca binária** no vetor ordenado (nada de tabela de dispersão: o vetor ordenado basta e tem desempenho igual nas seis linguagens).
3. Montar a árvore sobre os $m-1$ intervalos elementares. Seja N a menor potência de 2 com $N \geq m-1$; as folhas são os índices $N, N+1, \dots, 2N-1$ e o pai de um nó v é $\lfloor v/2 \rfloor$. Pré-calcular $\text{full}[v] =$ largura total do trecho do nó v : para a folha $N+k$ vale $y^{(k+1)} - y^{(k)}$ (ou 0 se $k \geq m-1$, folha de enchimento), e para um nó interno é a soma dos dois filhos. Zerar cover e len .
4. Montar os $2n$ eventos. Cada evento é uma **chave empacotada em um inteiro de 64 bits**: $(x \ll 18) \mid i$, onde i é o índice do evento (menor que 2^{18} , pois $2n \leq 10^5$). O evento de índice par é a borda esquerda do retângulo $i/2$ (soma +1) e o ímpar é a borda direita (soma -1). Ordenar o vetor de chaves — ordenar inteiros, não estruturas. A ordem entre eventos de mesmo x é irrelevante, porque a faixa entre eles tem largura zero.
5. **Varrer** os eventos em ordem, mantendo $\text{prev} =$ abscissa do evento anterior: acumular resposta $+= \text{len}[\text{raiz}] * (x - \text{prev})$, atualizar $\text{prev} = x$ e então aplicar o evento na árvore.
6. Imprimir a resposta.

A atualização $\text{update}(l, r, v)$ — somar $v \in \{+1, -1\}$ à cobertura das folhas l — é feita **de baixo para cima**, sem recursão:

1. Sejam $a = l + N$ e $b = r + N$ as folhas extremas. Enquanto $a < b$: se a é filho direito, o nó a é canônico, então $\text{cover}[a] += v$, recalcula-se $\text{len}[a]$ e faz-se $a \leftarrow a+1$; se b é filho direito, decrementa-se b , o nó b é canônico, $\text{cover}[b] += v$ e recalcula-se $\text{len}[b]$. Em seguida a e b sobem um nível ($a \leftarrow a/2, b \leftarrow b/2$).
2. Recalcular len subindo de $\lfloor (l+N)/2 \rfloor$ até a raiz, e depois subindo de $\lfloor (r+N-1)/2 \rfloor$ até a raiz. Todo nó canônico é filho de um nó de uma dessas duas cadeias, então essas duas subidas cobrem todos os ancestrais afetados. As duas cadeias são disjuntas abaixo do ancestral comum e coincidem acima dele; por isso a segunda subida — feita depois da primeira — corrige os nós compartilhados com os dois filhos já atualizados.

Recalcular len de um nó é sempre a mesma regra de três casos descrita acima, e é a única operação que escreve em len .

Complexidade

Seja $n \leq 5 \cdot 10^4$, com $2n \leq 10^5$ eventos e $m \leq 10^5$ coordenadas y distintas.

Tempo: $O(n \ln n)$. As três partes:

- ordenar os $2n$ valores de y e depois as $2n$ chaves de evento: $2 \cdot 10^5 \cdot 17 \approx 3,4 \times 10^6$ comparações no total;
- as $2n$ buscas binárias da compressão: $10^5 \cdot 17 \approx 1,7 \times 10^6$ passos;
- a varredura: $2n = 10^5$ atualizações na árvore. A árvore tem $N = 2^{17}$ folhas, ou seja 17 níveis acima das folhas; cada nível custa no máximo 2 nós canônicos mais 2 nós das cadeias de ancestrais, e cada nó custa um recálculo de len com no máximo duas leituras. São até $4 \cdot 17 \approx 68$ recálculos por atualização, isto é $10^5 \cdot 68 \approx 7 \times 10^6$ operações elementares.

O total fica na casa de 10^7 operações — folgado para o limite de tempo, e a medição confirma: a referência em C resolve o pior teste em cerca de 0,04 s.

Para comparação, a mesma varredura **sem** a árvore (mantendo a cobertura num vetor de contagem simples, pagando o tamanho do intervalo a cada evento) é $O(n \cdot m) = O(n^2)$: até 10^{10} operações, e mede 1,9 s no mesmo teste em que a solução com árvore leva 0,04 s — quase 50 vezes mais, muito além do limite.

Memória: $O(n)$. Os vetores da entrada e dos eventos somam $O(n)$ inteiros ($\approx 10^6$ bytes), e a árvore usa três vetores de $2N = 2^{18}$ inteiros de 32 bits (cover, len e full), cerca de 3 MB no total. Nada disso depende da magnitude das coordenadas, apenas da quantidade de retângulos — é por isso que a compressão de coordenadas é indispensável: sem ela a árvore teria 10^9 folhas.

Problema F — Fila de Julgamento

Solução

Ideia

Fixada uma ordem para a fila, a espera de cada submissão é uma **soma de prefixo**: se a ordem escolhida coloca os tempos na sequência $u_1, u_2, \dots, u_n$, a submissão que está na posição k tem espera $u_1 + u_2 + \dots + u_k$, e a soma total das esperas é

$$S = \sum_{k=1}^n \sum_{j=1}^k u_j = \sum_{k=1}^n (n-k+1) u_k.$$

A segunda forma é a observação-chave: o tempo que ficar na **posição** k é contado $n-k+1$ vezes — n vezes se for o primeiro da fila, uma vez só se for o último. Não existe ordem “sem custo”: o que dá para escolher é **quem paga o coeficiente grande**. Para minimizar, os coeficientes maiores têm de cair sobre os tempos menores, ou seja, a fila deve ser atendida em ordem **crescente** de tempo de julgamento (a regra clássica *shortest job first*).

Por que ordenar crescente é ótimo (argumento de troca). Tome uma ordem qualquer e suponha que ela tenha duas submissões **adjacentes** fora de ordem: nas posições k e $k+1$ estão os tempos a e b , com $a > b$. Seja P a soma dos tempos das submissões que vêm antes da posição k . Trocar essas duas de lugar não muda nada fora do par:

- quem está antes da posição k tem exatamente o mesmo prefixo, logo a mesma espera;
- quem está depois da posição $k+1$ enxerga o mesmo prefixo $P+a+b$ (a soma do par não muda), logo também mantém a espera.

Só as esperas dessas duas mudam. Antes da troca elas somam $(P+a)+(P+a+b)$; depois da troca somam $(P+b)+(P+b+a)$. A diferença é

$$[(P+a)+(P+a+b)] - [(P+b)+(P+b+a)] = a - b > 0,$$

ou seja, **a troca sempre melhora** (diminui a soma) quando o par está fora de ordem. Logo nenhuma ordem com um par adjacente fora de ordem pode ser ótima; e a única ordem sem nenhum par adjacente fora de ordem é a crescente. Empates não atrapalham: se $a = b$, a conta acima dá diferença 0, então tanto faz como se desempata — todas as ordens crescentes dão o mesmo valor, e por isso a resposta é única.

A abordagem ingênua — testar todas as ordens possíveis — é inviável: são $n!$ permutações, e já com $n=20$ isso passa de 2×10^{18} .

Cuidado com o tamanho da resposta. No pior caso ($n=2 \cdot 10^5$ submissões, todas com $t_i=10^6$) a soma vale $10^6 \cdot \frac{n(n+1)}{2} \approx 2,0 \times 10^{16}$: cabe folgado em um inteiro de 64 bits (long long, long, i64), mas estoura um inteiro de 32 bits, cujo máximo é 2147483647. O acumulador do prefixo também deve ser de 64 bits.

Passos

1. Ler n e o vetor $t_1, \dots, t_n$ (são até $2 \cdot 10^5$ números: use leitura rápida).

2. Ordenar o vetor t em ordem **crescente**, com o algoritmo de ordenação da biblioteca padrão.
3. Percorrer o vetor ordenado da esquerda para a direita mantendo duas variáveis de 64 bits, **acumulado** (a soma dos tempos já atendidos, inclusive o atual) e **total**: para cada i , fazer **acumulado** += $t[i]$ e em seguida **total** += **acumulado**.
4. Imprimir **total**.

Complexidade

O custo é dominado pela ordenação: **tempo** $O(n \log n)$. No pior caso anunciado, $2 \cdot 10^5 \cdot \log_2(2 \cdot 10^5) \approx 2 \cdot 10^5 \cdot 17,6 \approx 3,5 \times 10^6$ comparações, mais uma única passada de $2 \cdot 10^5$ somas para acumular o prefixo — sobra tempo em qualquer das linguagens aceitas.

A **memória** é $O(n)$: um vetor de $2 \cdot 10^5$ inteiros de 32 bits, cerca de 800 KB, mais dois acumuladores de 64 bits. Nada além da entrada precisa ser guardado, e nem sequer é necessário armazenar as esperas individuais — basta acumular a soma enquanto se percorre o vetor ordenado.

Problema G — Árvore de Mentores

Solução

Ideia

Chame de $S(r)$ a soma dos tempos de viagem de todos os treinadores até a sede r , ou seja, $S(r) = \sum_v d(r, v)$, onde $d(r, v)$ é a distância na árvore. A resposta é $m_r S(r)$.

O caminho ingênuo é calcular $S(r)$ para cada r com uma busca própria: n buscas de custo $O(n)$ dão $O(n^2)$, que para $n=150000$ é $2,25 \times 10^{10}$ operações — inviável. A saída é calcular S **de uma única vez** e depois **passear com a raiz**, aproveitando que dois vértices vizinhos têm respostas quase iguais.

A observação-chave. Enraíze a árvore em 1. Para um vértice u com pai p e aresta de peso w entre eles, chame de c_u o número de vértices da subárvore de u . Se a sede se mudar de p para u :

- os c_u treinadores que estão **dentro** da subárvore de u passam a viajar w minutos a **menos** cada um (deixaram de atravessar a aresta (p, u));

- os outros $n - c_u$ treinadores passam a viajar w minutos a **mais** cada um.

Logo

$$S(u) = S(p) - c_u \cdot w + (n - c_u) \cdot w = S(p) + (n - 2c_u) \cdot w.$$

Essa é a recorrência inteira do problema: sabendo S do pai e o tamanho da subárvore do filho, o S do filho sai em **tempo constante**. Note que o fator $(n - 2c_u)$ é negativo exatamente quando mais da metade dos treinadores está na subárvore de u — é o sinal de que vale a pena empurrar a sede naquela direção.

Falta só o ponto de partida $S(1)$. Ele vem de uma soma de baixo para cima: se s_u é a soma das distâncias de u até os vértices **da sua própria subárvore**, então cada filho f de u , ligado por uma aresta de peso w_f , contribui com o que já era interno à subárvore de f mais uma travessia da aresta para cada um dos c_f vértices de lá:

$$s_u = \sum_{f \text{ filho de } u} (s_f + c_f \cdot w_f), \quad c_u = 1 + \sum_{f \text{ filho de } u} c_f.$$

Como s_1 soma as distâncias de 1 até a árvore toda, $S(1) = s_1$.

Como fazer as duas varreduras sem recursão. Uma busca em largura a partir de 1 produz um vetor `ordem[]` em que todo vértice aparece **depois** do seu pai. Percorrer `ordem[]` de trás para frente é, portanto, uma ordem topológica de baixo para cima (todo filho já foi processado quando se chega ao pai), e percorrê-lo para frente é a ordem de cima para baixo. Duas varreduras lineares sobre o mesmo vetor resolvem tudo, sem pilha de recursão — o que importa porque a árvore pode ser uma linha reta com 150 000 vértices, profundidade que estoura a pilha de várias linguagens.

A lista de adjacência é montada em **CSR** (três vetores planos: conta os graus, faz a soma de prefixo para achar onde começa cada vértice, e preenche), evitando estrutura dinâmica por vértice.

Um atalho equivalente (para quem enxergar antes). A mesma recorrência mostra que mudar a sede de p para u só compensa quando $n - 2c_u < 0$, isto é, quando **mais da metade** dos treinadores está do lado de u . Logo o ótimo é o **centroide** da árvore — o vértice em que nenhum ramo tem mais de $n/2$ vértices — e ele não depende dos pesos das arestas. Quem perceber isso pode achar o centroide com uma varredura de tamanhos de subárvore e responder com **uma única** busca a partir dele, também em $O(n)$. As duas soluções são igualmente aceitáveis; a troca de raiz é

apresentada aqui por ser a técnica reaproveitável (serve para qualquer acumulado que dependa da raiz, não só para soma de distâncias).

Cuidado com o tamanho da resposta. No pior caso — uma linha reta com 150000 vértices e todas as arestas custando 10^6 — a melhor soma já é $10^6 \cdot 2 \cdot \frac{75000 \cdot 75001}{2} \approx 5,6 \times 10^{15}$, e a soma da ponta chega a $\approx 1,1 \times 10^{16}$. Isso não cabe em 32 bits: tanto a resposta quanto os acumuladores s_u e $S(u)$ precisam de inteiros de **64 bits**. Os tamanhos de subárvore c_u cabem em 32 bits, mas o produto $c_u \cdot w$ tem de ser feito em 64 bits.

Passos

1. Ler n e as $n-1$ arestas (a, b, w) , contando o grau de cada vértice.
2. Montar a lista de adjacência em CSR: soma de prefixo dos graus para obter `inicio[v]`, depois uma passada preenchendo `vizinho[]` e `peso[]`.
3. Fazer uma busca em largura a partir do vértice 1, gravando `ordem[]` (a ordem de visita), `pai[v]` e `pesoAoPai[v]`.
4. Inicializar $c_v=1$ e $s_v=0$ para todo v e percorrer `ordem[]` **de trás para frente**; para cada $u \neq 1$, com $p=\text{pai}[u]$ e $w=\text{pesoAoPai}[u]$, fazer $c_p + c_u$ e $s_p + s_u + c_u \cdot w$.
5. Fazer $S(1)=s_1$ e percorrer `ordem[]` **para frente**; para cada $u \neq 1$, calcular $S(u)=S(\text{pai}[u])+(n-2c_u) \cdot \text{pesoAoPai}[u]$.
6. Imprimir o menor valor de S encontrado.

Complexidade

Cada uma das quatro passadas — leitura, montagem do CSR, busca em largura e as duas varreduras — percorre os vértices e as arestas um número constante de vezes: **tempo** $O(n)$. No limite $n=150000$ são $2(n-1)=299998$ entradas de adjacência visitadas uma vez na construção e uma vez na busca, mais $2n=300000$ atualizações nas duas varreduras, ou seja da ordem de $1,2 \times 10^6$ operações elementares — sobra folga enorme. O custo dominante, na prática, é ler os $1+3(n-1)=449998$ inteiros da entrada (cerca de 3 MB), e é por isso que vale usar leitura em bloco em vez de leitura formatada campo a campo.

A **memória** é $O(n)$: os três vetores de arestas, os dois vetores do CSR de tamanho $2(n-1)$ e meia dúzia de vetores de tamanho n (`ordem`, `pai`, `peso ao pai`, `tamanho de subárvore`, e dois de 64 bits para s e S) somam cerca de 12 MB para $n=150000$ — muito abaixo do limite. Nenhuma pilha de recursão é usada, então a profundidade da árvore não influi.

Para comparação: a solução ingênua, com uma busca a partir de cada

vértice, faz $n \cdot O(n) = 2,25 \times 10^{10}$ operações no maior caso, ordens de grandeza acima do que cabe no tempo-limite.

Problema H — Penalidade

Solução

Ideia

A regra parece ter três cláusulas independentes, mas todas falam da mesma coisa: **o que importa de cada problema é o que aconteceu até o primeiro acerto dele**. Depois do acerto, o problema está encerrado; antes do acerto, cada erro vale 20 minutos — mas só se o acerto **vier**.

Isso sugere guardar, para cada um dos k problemas, apenas duas informações:

- `resolvido[p]`: se a equipe já acertou o problema p ;
- `erros[p]`: quantas submissões erradas a equipe fez em p **desde o começo até agora**, sem contar as que vieram depois do acerto.

Com esses dois vetores, uma única passada pela lista de submissões resolve tudo, porque a entrada já chega em ordem cronológica: quando o acerto de p aparece, `erros[p]` é exatamente o número de erros que precederam esse acerto, e essa é a hora — a única hora — de cobrar a multa. A contribuição de p é fechada ali: $m + 20 \cdot \text{erros}[p]$.

As três armadilhas do enunciado caem sozinhas nesse desenho:

- **erro que não conta**: se um problema nunca é acertado, ninguém nunca lê o `erros[p]` dele, e os erros somem da conta. Não é preciso “descontar” nada no fim, é preciso **não somar** durante o caminho. Quem soma 20 a cada erro que lê e tenta corrigir depois erra os testes em que o time espalhou erros por problemas que não fechou.
- **submissão depois do acerto**: basta que a primeira coisa feita a cada linha seja testar `resolvido[p]` e pular a submissão. Sem esse teste, um segundo acerto no mesmo problema conta o problema duas vezes e cobra o minuto de novo.
- **acerto no minuto 0**: contribui $0 + 20 \cdot \text{erros}[p]$, que pode ser 0. Uma penalidade zero com problemas resolvidos é uma resposta legítima, e não um sinal de que faltou processar alguma coisa.

Não há nada a otimizar além disso: qualquer solução tem de olhar as n submissões pelo menos uma vez, e uma passada basta. O caminho ingênuo que existe aqui não é o de tempo, é o de **memória e de ordem**: guardar a lista inteira de submissões de cada problema para depois procurar o acerto

e contar os erros anteriores funciona, mas gasta $O(n)$ de memória e uma segunda varredura sem necessidade nenhuma — os dois inteiros por problema já carregam todo o passado que interessa.

Como $k \leq 14$, os dois vetores cabem em algumas dezenas de bytes; o tamanho da entrada não influencia a memória de trabalho.

Passos

1. Ler n e k .
2. Zerar os vetores `resolvido[1..k]` e `erros[1..k]`, e as duas respostas: `total = 0` (problemas resolvidos) e `pen = 0` (penalidade, em 64 bits).
3. Repetir n vezes, lendo p , m e v de cada submissão, nesta ordem:
 1. se `resolvido[p]` é verdadeiro, ignorar a submissão e passar para a próxima;
 2. senão, se $v=1$: marcar `resolvido[p]`, somar 1 em `total` e somar $m+20 \cdot \text{erros}[p]$ em `pen`;
 3. senão ($v=0$): somar 1 em `erros[p]`.
4. Imprimir `total` e `pen` separados por um espaço.

Note que os vetores são indexados de 1 a k (alocar $k+1$ posições evita subtrair 1 do p lido e é uma fonte a menos de erro de índice).

Complexidade

O laço principal executa exatamente n iterações e cada iteração faz um número constante de operações: uma leitura de três inteiros, um teste, e uma soma. A inicialização dos vetores custa $O(k)$. Logo o **tempo é $O(n+k)$** — no pior caso $10^5+14 \approx 10^5$ iterações, algo como $3 \cdot 10^5$ inteiros lidos, que é o custo real do programa: a leitura da entrada domina tudo, e por isso vale usar leitura em bloco (`fread/read().split()/DataInputStream`) em vez de uma chamada de leitura formatada por número.

A **memória é $O(k)$** de trabalho — dois vetores de no máximo 15 posições, menos de 200 bytes — porque nenhuma submissão precisa ser guardada depois de processada. A entrada é consumida em fluxo.

Sobre o tipo da penalidade: o máximo possível sai de resolver todos os $k=14$ problemas no minuto 300 gastando todas as demais submissões como erros anteriores, ou seja $14 \cdot 300 + 20 \cdot (10^5 - 14) = 4200 + 1999720 = 2003920$. Isso cabe folgado até em 32 bits, mas a conta é de acumulação e o hábito certo em problema de placar é acumular em 64 bits (`long long`, `long`, `i64`), que aqui não custa nada.

Problema I — Escala dos Juízes

Solução

Ideia

O que o enunciado pede é o **problema da atribuição**: dado um grafo bipartido completo com n juízes de um lado, n problemas do outro e custo c_{ij} em cada aresta, achar o **emparelhamento perfeito de custo mínimo**.

Três abordagens naturais morrem antes de começar:

- **testar todas as escalas**: são $n!$ permutações, e $200!$ tem mais de 370 algarismos;
- **programação dinâmica sobre subconjuntos de problemas** ($O(2^n \cdot n)$), a solução padrão quando $n \leq 20$: aqui seriam 2^{200} estados;
- **guloso** (dar a cada juiz o problema mais barato ainda livre): simplesmente erra, como mostra o primeiro exemplo — o juiz barato em tudo tem de ser poupado para o problema em que os outros são caros.

A saída certa vem de enxergar a escala como um **fluxo**. Monte a rede: uma fonte s ligada a cada juiz com capacidade 1 e custo 0; cada juiz i ligado a cada problema j com capacidade 1 e custo c_{ij} ; cada problema ligado ao sorvedouro t com capacidade 1 e custo 0. Um fluxo inteiro de valor n é uma escala válida, e o custo do fluxo é o desgaste total. Como todas as capacidades são inteiras, existe fluxo ótimo inteiro — logo, o **fluxo de custo mínimo de valor n** dá exatamente a resposta.

Para calcular esse fluxo usamos **caminhos mínimos sucessivos**: mande 1 unidade de cada vez pelo caminho aumentante mais barato. O teorema que sustenta o método diz que, se cada unidade vai pelo caminho mais barato, o fluxo obtido depois de k unidades é ótimo entre todos os fluxos de valor k — então depois de n unidades temos a escala ótima.

Falta um detalhe incômodo: no grafo residual aparecem arestas de custo **negativo** (as arestas de volta, com custo $-c_{ij}$), e Dijkstra não funciona com peso negativo. A saída clássica são os **potenciais de Johnson**: mantemos u_i para cada juiz e v_j para cada problema e trabalhamos com o **custo reduzido** $c_{ij} - u_i - v_j$, que o algoritmo mantém sempre ≥ 0 (e igual a 0 nas arestas já usadas pela escala corrente). Com custo reduzido não-negativo, Dijkstra é válido; ao fim de cada busca somamos as distâncias aos potenciais e a invariante se preserva.

O último passo é escolher **qual** Dijkstra. O grafo é **denso** (n^2 arestas), e num grafo denso a versão certa é a de $O(V^2)$ — a que escolhe o próximo vértice por varredura linear, sem heap. Fazendo essa escolha, a busca por juiz vira o que se conhece como **algoritmo húngaro** na formulação de Jonker-Volgenant, e é ela que está implementada nas soluções de referência: para cada juiz i cresce-se uma árvore alternante sobre as colunas, guardando em $\text{minv}[j]$ o menor custo reduzido de dentro da árvore até a coluna j ainda de fora, e deslocando os potenciais de delta (o menor minv) a cada passo. Quando a árvore alcança um problema ainda sem dono, o vetor $\text{way}[]$ guarda o caminho alternante e basta invertê-lo para aumentar a escala em um juiz.

Note por que o enunciado pede **só o total**: quando há empates (segundo exemplo) várias escalas diferentes atingem o mínimo, mas o valor mínimo é único.

Sobre tipos: a resposta chega a $200 \times 10^6 = 2 \times 10^8$, que ainda cabe em 32 bits, mas os potenciais, a sentinela de “infinito” e as somas intermediárias não têm essa garantia tão folgada — as seis referências usam inteiros de 64 bits em tudo.

Passos

1. Ler n e a matriz c num vetor plano de $(n+1)^2$ inteiros de 64 bits (índices de 1 a n , para casar com a formulação clássica).
2. Zerar os potenciais u e v e o vetor p , em que $p[j]$ guarda o juiz que ficou com o problema j (0 = problema ainda sem dono).
3. Para cada juiz i de 1 a n , executar uma fase:
 1. pôr $p[0] = i$ e começar a árvore alternante em $j_0 = 0$; iniciar $\text{minv}[j]$ com infinito e $\text{used}[j]$ com falso;
 2. marcar j_0 como usado e, com $i_0 = p[j_0]$, percorrer todos os problemas j ainda não usados atualizando $\text{minv}[j]$ com $c_{i_0 j} - u_{i_0} - v_j$ (guardando em $\text{way}[j]$ de onde veio a melhora) e anotando o menor minv , chamado delta, e o problema j_1 onde ele ocorre;
 3. deslocar os potenciais: para todo j usado, $u[p[j]] += \text{delta}$ e $v[j] -= \text{delta}$; para todo j não usado, $\text{minv}[j] -= \text{delta}$. Isso mantém o custo reduzido não-negativo e zera a aresta que acabou de entrar na árvore;
 4. avançar $j_0 = j_1$; se $p[j_0]$ ainda for 0, o problema j_1 não tem dono e o caminho aumentante terminou — sair do laço; senão, voltar ao passo 3.2;
 5. inverter o caminho alternante andando por $\text{way}[]$ de volta até a raiz: $p[j_0] = p[\text{way}[j_0]]$, $j_0 = \text{way}[j_0]$, até chegar em 0.
4. Somar $c_{p[j]j}$ para j de 1 a n e imprimir esse total.

Complexidade

São n fases (uma por juiz). Cada fase faz no máximo n passos de crescimento da árvore, e cada passo varre os n problemas duas vezes (uma para atualizar minv , outra para deslocar os potenciais): $O(n)$ por passo, $O(n^2)$ por fase, $O(n^3)$ **no total**. Com $n=200$ isso dá $200^3=8\times 10^6$ operações elementares no pior caso — sobra tempo em qualquer das seis linguagens, inclusive no PyPy.

A memória é dominada pela matriz: $(n+1)^2=40401$ inteiros de 64 bits, cerca de 320 KB, mais uns poucos vetores de tamanho $n+1$. Ou seja, $O(n^2)$ **de memória**, com $O(n)$ de estrutura auxiliar. A leitura são $n^2=40000$ números (aproximadamente 280 KB de texto), então vale a pena usar leitura em buffer.

Um contraste que explica a escolha do Dijkstra denso: um fluxo de custo mínimo genérico, com Dijkstra sobre heap binário, gastaria $O(n \cdot E \log V)$ operações de heap, ou seja $200 \cdot 40000 \cdot \log_2(400) \approx 1,4 \times 10^8$ — umas **vinte vezes** mais caro que a varredura linear, num grafo em que toda aresta existe. Denso pede varredura, não heap.

Já uma implementação que recalcula o mínimo do zero a cada passo, sem manter minv incrementalmente, custa $O(n^4)=1,6 \times 10^9$ e não termina no tempo (é a solução de controle em `sols/slow/tle.cpp`).

Problema J — Termostato do Sabiazinho

Solução

Ideia

O termostato responde a cada ajuste com uma informação binária: FRIO se o valor perguntado é maior ou igual ao limiar T , QUENTE se é menor. Isso é exatamente o oráculo de uma **busca binária** sobre o intervalo $[1, 10^9]$, e o limite de 30 perguntas conta a história inteira: $\lceil \log_2 10^9 \rceil = 30$, então a busca binária cabe — e **nada com folga**. Trinta é o mínimo teórico-informacional para distinguir 10^9 valores com respostas binárias: cada pergunta divide o conjunto de candidatos em dois, e $2^{29} < 10^9 \leq 2^{30}$. Não existe estratégia correta com menos perguntas no pior caso, e o painel não perdoa nenhuma pergunta desperdiçada.

Mantém-se um intervalo de candidatos $[l, h]$ que contém T com certeza.

Pergunta-se o meio $m = \lfloor (l o + h i) / 2 \rfloor$: resposta FRI0 significa $T \leq m$ e o intervalo vira $[l o, m]$; resposta QUENTE significa $T > m$ e o intervalo vira $[m+1, h i]$. Quando $l o = h i$, esse valor é T e a linha final ! T — que não conta como pergunta — encerra.

O cuidado que decide o problema é **não gastar pergunta à toa**. Três formas clássicas fecham em exatamente 30 no pior caso e passam sempre:

- while (lo < hi) com $m = (lo+hi)/2$, FRI0 -> $hi = m$, QUENTE -> $lo = m+1$, respondendo lo;
- a forma com resposta acumulada: while (lo <= hi) guardando em ans o último FRI0 visto;
- a forma simétrica que estreita por hi, com os mesmos papéis trocados.

Já a variante — correta! — que mantém um invariante aberto while (hi - lo > 1) e, ao final, **sonda** lo com uma pergunta extra para confirmar, usa 31 perguntas no pior caso e morre no painel travado com *Wrong Answer* na maioria dos limiares. A confirmação é redundante: quando o intervalo tem um único candidato, ele é T , porque o invariante já garantia que T estava dentro. A resposta final não precisa, e não deve, ser precedida de teste.

Dois detalhes de protocolo completam a solução. Primeiro, o *flush*: a conversa passa por um *pipe*, e uma pergunta presa no buffer de saída nunca chega ao juiz — o programa espera uma resposta que não virá e o veredito degenera em *Time Limit Exceeded*. Todo ? x precisa ser descarregado (fflush(stdout), std::endl, flush=True, System.out.flush(), io::stdout().flush()). Segundo, a leitura: depois de cada pergunta lê-se **uma** resposta, antes de perguntar de novo; desalinhar pergunta e resposta trava a conversa dos dois lados.

Passos

1. Inicializar $l o = 1$ e $h i = 10^9$.
2. Enquanto $l o < h i$: perguntar ? m com $m = \lfloor (l o + h i) / 2 \rfloor$ (e dar *flush*); ler a resposta; se FRI0, fazer $h i = m$; se QUENTE, fazer $l o = m + 1$.
3. Imprimir ! lo (com *flush*) e encerrar — sem pergunta de confirmação.

Complexidade

São no máximo $\lceil \log_2 10^9 \rceil = 30$ iterações, cada uma com uma pergunta, uma leitura e aritmética constante: $O(\log T_m)$ **perguntas e tempo**, memória $O(1)$. O pior caso real é $T=1$: a cadeia de respostas FRI0 estreita o intervalo por cima e consome as 30 perguntas inteiras, sem sobrar nenhuma. Em C, C++, Java, Kotlin, Python e Rust o custo de máquina é desprezível; o problema mede exclusivamente a disciplina da busca e do protocolo.

Problema K — O Primeiro Balão

Solução

Ideia

Não há nada a calcular: a resposta é uma função de duas variáveis que só assumem 302 valores cada. O problema todo está em **ordenar as perguntas** — e em não deixar o -1 entrar na comparação de minutos.

A armadilha é essa: -1 **não** é um minuto. É uma marca de “não resolveu”, e por acaso ela é representada pelo menor número da entrada. Quem escreve direto

```
se a < b entao A senao B
```

responde A para a linha $-1\ 12$, ou seja, entrega o balão para uma equipe que não resolveu o problema. O menor número só decide alguma coisa **depois** de ficar estabelecido que os dois números são minutos de verdade.

Daí sai a ordem correta de perguntas, da mais grosseira para a mais fina:

1. **as duas resolveram?** Se nenhuma resolveu ($a = -1$ e $b = -1$), a resposta é NINGUEM e o balão fica na caixa;
2. **só uma resolveu?** Então é dela, sem olhar o minuto: um Accepted no minuto 300 ganha de um problema não resolvido;
3. **as duas resolveram.** Agora, e só agora, o menor minuto vence; se os minutos forem iguais, é EMPATE.

Uma tentação comum é tentar eliminar os casos especiais trocando o -1 por “infinito” (por exemplo, 10^9) e comparando. Funciona para os três primeiros casos, mas **quebra no quarto**: com as duas equipes marcadas como infinito, os valores ficam iguais e o programa responde EMPATE onde a resposta é NINGUEM. Se você usar esse truque, o teste de “nenhuma resolveu” tem de ser tratado **antes** — que é exatamente a ordem descrita acima.

Vale conferir que as regras cobrem tudo e não se sobrepõem: as quatro perguntas particionam os 302×302 pares possíveis, e cada par cai em exatamente um ramo, porque o último senão é irrestrito. A resposta, portanto, é única.

Passos

1. Ler os dois inteiros a e b da única linha da entrada.
2. Se $a = -1$ e $b = -1$, imprimir NINGUEM e encerrar.
3. Senão, se $b = -1$ (logo a é um minuto), imprimir A e encerrar.

4. Senão, se $a = -1$ (logo b é um minuto), imprimir B e encerrar.
5. Senão, as duas resolveram: se $a < b$, imprimir A; se $b < a$, imprimir B; caso contrário ($a = b$), imprimir EMPATE.

Nenhum laço, nenhum vetor, nenhuma conversão de tipo delicada: a e b cabem folgadoamente em um inteiro de 32 bits, e o programa só faz comparações — não há soma nem produto, logo não há como estourar nada.

Complexidade

O programa lê dois inteiros e avalia **no máximo 6 comparações**, sem laço e sem estrutura de dados: **tempo** $O(1)$ e **memória** $O(1)$, com constante minúscula e independente da entrada. O número de operações no pior caso (a linha 74 74, que percorre a cadeia inteira) é um punhado de instruções; o custo real do programa é a leitura de uma linha de no máximo 8 caracteres.

Problema L — Placar ao Vivo

Solução

Ideia

O caminho ingênuo é manter o placar num vetor $p[1..n]$: uma soma em faixa percorre de l a r somando x , e uma consulta percorre de l a r pegando o maior. Cada evento custa $O(n)$, e com $n = q = 2 \cdot 10^5$ isso dá $4 \cdot 10^{10}$ operações no pior caso — inviável.

A estrutura que resolve os dois lados de uma vez é a **segment tree**. Ela é uma árvore binária sobre os índices: a raiz cuida de $[1, n]$, cada nó interno parte o seu segmento ao meio e as folhas cuidam de uma equipe cada. Guardamos em $mx[nd]$ o maior valor do segmento do nó nd . Qualquer faixa $[l, r]$ se decompõe em $O(\lg n)$ nós cujos segmentos são **inteiramente** contidos nela, então uma consulta de máximo custa $O(\lg n)$.

O problema é a soma em faixa: atualizar folha por folha voltaria ao custo $O(n)$. A saída é a **lazy propagation**. Quando o segmento de um nó cabe inteiro dentro de $[l, r]$, todo mundo embaixo dele recebe exatamente o mesmo x — e o máximo daquele segmento simplesmente sobe x . Então paramos ali: fazemos $mx[nd] += x$ e anotamos a dívida em $lz[nd] += x$, sem descer. A convenção que usamos é a mais econômica possível:

- $mx[nd]$ já é o máximo verdadeiro do segmento de nd , contando tudo o que foi aplicado **em** nd **ou abaixo dele** (mas não o que está pendente nos ancestrais);

- $lz[nd]$ é o quanto foi somado ao segmento inteiro de nd sem ter sido repassado aos filhos.

Com essa convenção **não existe operação de empurrar a dívida para baixo**. Na atualização, ao voltar de uma divisão, basta recompor $mx[nd] = \max(mx[2nd], mx[2nd+1]) + lz[nd]$. Na consulta, descemos carregando acc , a soma dos lz dos ancestrais já visitados; quando um nó está totalmente coberto, o candidato a resposta é $mx[nd] + acc$. Como o valor de um nó nunca é reescrito por causa de uma consulta, a árvore continua consistente.

Duas armadilhas de implementação valem mais do que a ideia:

- **A resposta não cabe em 32 bits.** São até $2 \cdot 10^5$ desafios de até 10^6 pontos caindo na mesma equipe: $2 \cdot 10^{11}$, cerca de cem vezes o teto de um inteiro com sinal de 32 bits (2147483647). O acumulador, o mx , o lz e o acc têm de ser de 64 bits.
- **A entrada é grande.** São até $8 \cdot 10^5$ números para ler e até $2 \cdot 10^5$ linhas para escrever. Leitura formatada padrão e uma impressão por linha custam mais do que o algoritmo inteiro: leia o buffer de uma vez e acumule a saída, imprimindo tudo no fim.

Nas soluções de referência a árvore mora em dois vetores planos de tamanho $4n$ e a descida é feita com uma **pilha explícita** em vetores, em vez de chamadas recursivas — isso mantém as seis linguagens como transliterações literais umas das outras e evita o custo (e o limite) de pilha de recursão. A pilha nunca guarda mais do que umas poucas dezenas de entradas, porque em cada nível da árvore no máximo dois nós ficam parcialmente cobertos.

Passos

1. Ler n e q ; criar $mx[1..4n]$ e $lz[1..4n]$ zerados (o placar começa todo em 0, então a árvore zerada já é a árvore correta — não é preciso construí-la).
2. Para cada evento, ler o tipo e a faixa l, r .
3. Se o tipo é 1, ler x e chamar a atualização, que percorre a árvore a partir da raiz:
 - segmento do nó disjunto de $[l, r]$: ignora;
 - segmento do nó contido em $[l, r]$: $mx[nd] += x$, $lz[nd] += x$ e **para** (é a preguiça);
 - caso contrário: desce nos dois filhos e, na volta, refaz $mx[nd] = \max(mx[2nd], mx[2nd+1]) + lz[nd]$.
4. Se o tipo é 2, chamar a consulta, que percorre a árvore a partir da raiz carregando acc (soma dos lz dos ancestrais, começando em 0):
 - segmento disjunto de $[l, r]$: ignora;
 - segmento contido em $[l, r]$: candidato $mx[nd] + acc$, guarda o maior

visto e **para**;

- caso contrário: desce nos dois filhos com $\text{acc} + \text{lz}[\text{nd}]$. Anexar o máximo obtido ao buffer de saída.

5. Ao fim dos q eventos, imprimir o buffer de saída de uma vez só.

A “volta da recursão” do passo 3 é emulada empilhando o nó uma segunda vez, marcado com uma fase diferente: desempilhado nessa fase, ele só recompõe $\text{mx}[\text{nd}]$ a partir dos filhos, que já foram totalmente processados.

Complexidade

A árvore tem altura $\lceil \lg n \rceil \leq 18$ para $n \leq 2 \cdot 10^5$. Tanto a atualização quanto a consulta visitam no máximo 4 nós por nível (em cada nível, no máximo dois nós ficam parcialmente cobertos, e cada um deles gera dois filhos), ou seja, $O(\lg n)$ nós por evento — na prática, menos de 72 nós visitados. Com $q \leq 2 \cdot 10^5$ eventos, o total fica em torno de $2 \cdot 10^5 \cdot 72 \approx 1,4 \times 10^7$ visitas a nós, cada uma com um punhado de comparações e somas de 64 bits. Somando a leitura dos $8 \cdot 10^5$ números da entrada, isso roda em poucas dezenas de milissegundos numa linguagem compilada.

O **tempo total** é $O((n+q)\lg n)$ e a **memória** é $O(n)$: dois vetores de $4n$ inteiros de 64 bits, ou seja, $2 \cdot 8 \cdot 10^5 \cdot 8 \approx 12,8$ MB para o n máximo, mais o buffer de saída de até $2 \cdot 10^5$ linhas. A pilha explícita ocupa espaço constante — bastam algumas dezenas de entradas, porque a profundidade da descida é limitada pela altura da árvore.

Problema M — Linhagem dos Treinadores

Solução

Ideia

O caminho entre dois vértices de uma árvore é único, e ele tem um formato fixo: sobe de a até o **ancestral comum mais profundo** de a e b — o $\text{lca}(a, b)$ — e desce até b . Percorrer esse caminho com o dedo custa $O(n)$ por consulta; com q consultas isso vai a $2 \cdot 10^5 \cdot 5 \cdot 10^4 = 10^{10}$ passos no pior caso (o painel em linha reta), o que não fecha em tempo nenhum. A saída é responder cada consulta em tempo logarítmico, e para isso bastam duas ferramentas.

Primeira: soma de prefixo na árvore. Defina $S[x]$ como a soma dos v de todos os vértices no caminho da raiz até x , incluindo os dois. Como o mentor p_x é o vizinho de x nesse caminho, vale $S[x] = S[p_x] + v_x$, com

$S[1]=v_1$. Um detalhe do enunciado faz esse cálculo ser quase de graça: como $p_i < i$, quando o laço chega em i o valor $S[p_i]$ **já foi calculado**. Ou seja, a entrada já vem em ordem topológica e um laço simples de $i=2$ até n resolve — não é preciso nem BFS nem recursão (que, aliás, estouraria a pilha num painel em linha reta com $2 \cdot 10^5$ vértices). A mesma observação dá a profundidade: $d e p[x] = d e p[p_x] + 1$.

Segunda: a fórmula do caminho. Seja $l = lca(a, b)$. A soma do caminho de a até b é

$$S[a] + S[b] - 2 \cdot S[l] + v_l.$$

Por quê? $S[a]$ cobre raiz $\rightarrow a$ e $S[b]$ cobre raiz $\rightarrow b$. O trecho raiz $\rightarrow l$ aparece nas duas somas, então tem de ser descontado **duas** vezes: é o $-2 \cdot S[l]$. Só que $S[l]$ inclui o próprio l , que pertence ao caminho e foi descontado a mais — daí o $+v_l$ de volta. O caso $a=b$ cai na mesma fórmula sem tratamento especial: $l=a$ e sobra exatamente v_a .

Como achar o LCA rápido: binary lifting. Guarde $up[k][x]$ = o ancestral de x que está 2^k níveis acima (ou 0, um vértice fantasma, se não existir). A tabela sai de si mesma: $up[0][x] = p_x$ e $up[k][x] = up[k-1][up[k-1][x]]$ — de novo um laço, sem recursão. Com ela, subir d níveis é olhar os bits de d (se o bit k está ligado, pule 2^k), e o LCA de a e b sai em dois passos: primeiro iguale as profundidades, e depois, se ainda forem vértices diferentes, desça k de 17 até 0 subindo os dois juntos **somente** enquanto os ancestrais forem diferentes. No fim, a e b param exatamente um nível abaixo do LCA, e a resposta é $up[0][a]$.

Como $2^{17} = 131072 < 2 \cdot 10^5 \leq 2^{18}$, 18 níveis ($k=0 \dots 17$) cobrem qualquer profundidade possível.

Um cuidado de tipo fecha o problema: o painel pode ser uma linha reta com $2 \cdot 10^5$ treinadores de $v_i = 10^6$ cada, e aí a resposta de uma única consulta é $2 \cdot 10^{11}$ — muito acima dos 2147483647 de um inteiro de 32 bits. Tanto os $S[x]$ quanto a resposta precisam de 64 bits (long long, long, i64).

Passos

1. Ler n , q , o vetor v e os mentores $p_2, \dots, p_n$ (a leitura tem até $5 \cdot 10^5$ números: use leitura rápida em bloco).
2. Num laço de $i=2$ até n , calcular $d e p[i] = d e p[p_i] + 1$ e $S[i] = S[p_i] + v_i$, com $d e p[1] = 0$ e $S[1] = v_1$. A ordem já está garantida por $p_i < i$.

3. Preencher $up[0][x] = p_x$ (com $up[0][1] = 0$) e, para k de 1 a 17, preencher $up[k][x] = up[k-1][up[k-1][x]]$ para todo x , mantendo $up[k][0] = 0$.
4. Para cada consulta (a, b) : garantir que a é o mais profundo (trocando os dois se preciso), subir a em $dep[a] - dep[b]$ níveis pelos bits, e — se ainda $a \neq b$ — subir os dois juntos de $k=17$ até 0 enquanto $up[k][a] \neq up[k][b]$. O LCA é a (se os dois se encontraram na primeira fase) ou $up[0][a]$.
5. Acumular $S[a] + S[b] - 2 \cdot S[l] + v_l$ em 64 bits num buffer de saída e imprimir tudo de uma vez no fim.

Complexidade

A construção é dominada pela tabela de binary lifting: $18 \times 2 \cdot 10^5 = 3,6 \times 10^6$ atribuições, mais dois laços lineares para dep e S . Cada consulta faz no máximo 18 saltos para igualar as profundidades e mais 18 para achar o LCA, ou seja, $5 \cdot 10^4 \times 36 = 1,8 \times 10^6$ operações no total. O tempo é $O((n+q) \lg n)$, cerca de $5,4 \times 10^6$ operações no pior caso — folgado.

A memória é $O(n \lg n)$: a tabela up é um vetor plano de $18 \times (n+1) \approx 3,6 \times 10^6$ inteiros de 32 bits, uns 14 MB, mais $O(n)$ para v , dep e S — bem dentro do limite de 1 GB. Não vale a pena trocar a tabela por uma sparse table de *Euler tour*: ela usa o dobro de memória para a mesma complexidade.

Problema N — Kit do Competidor

Solução

Ideia

Este é o problema da **mochila 0/1** na forma mais direta: n objetos, cada um com um peso w_i e um valor v_i , uma capacidade W , e cada objeto pode ser levado **no máximo uma vez**. Queremos o maior valor total com peso total até W .

Duas tentações que **não** funcionam:

- **Testar todos os subconjuntos.** São 2^n combinações; com $n=1000$ isso é 2^{1000} , um número maior que a quantidade de átomos do universo observável. Inviável.
- **Ser guloso pela densidade** v_i/w_i , pegando primeiro os brindes que “rendem mais por grama”. Isso resolve a mochila *fracionária* (aquela

em que dá para levar meia caneca), mas não a nossa: como o brinde é indivisível, sobra um pedaço de sacola que o guloso não sabe aproveitar. O segundo exemplo é exatamente esse contraexemplo — o guloso leva o brinde de 300 g (o de maior densidade) e depois só consegue encaixar o de 200 g, fechando 8 pontos, enquanto a escolha ótima é largar o primeiro e levar os de 400 g e 200 g, que fecham 9.

A observação-chave é que, para decidir o que fazer com o brinde i , **a única informação que importa do passado é quanto peso ainda cabe na sacola**. Não importa *quais* brindes já entraram, só o espaço que sobrou. Isso dá o estado da programação dinâmica:

$d\ p[j]$ = maior soma de pontos usando apenas os brindes já processados, com peso total $\leq j$.

Processando um brinde novo (w, v) , cada capacidade $j \geq w$ tem duas opções: não levar o brinde (fica o $d\ p[j]$ antigo) ou levar (fica $d\ p[j-w] + v$, valor antigo da capacidade reduzida). Ou seja, $d\ p[j] \leftarrow \max(d\ p[j], d\ p[j-w] + v)$.

Isso poderia virar uma tabela bidimensional de $n \times (W+1)$ células, mas repare que a linha do brinde i só olha para a linha do brinde $i-1$. Dá para manter **um único vetor** de tamanho $W+1$ e atualizá-lo no lugar — desde que a varredura de j seja feita **de trás para frente**, de W até w . O motivo é sutil e é o ponto onde a maioria dos erros acontece:

- varrendo **para trás**, quando a posição j é atualizada, a posição $j-w$ ainda não foi tocada nesta rodada: ela guarda o valor *sem* o brinde atual. Cada brinde entra no máximo uma vez, que é o que o enunciado pede;
- varrendo **para a frente**, $d\ p[j-w]$ já teria sido atualizada com o brinde atual, e o mesmo brinde poderia ser somado várias vezes. Esse laço resolve outro problema (a mochila com repetição ilimitada) e dá resposta maior que a correta — no terceiro exemplo, por exemplo, levaria duas vezes o brinde de 500 g.

Como todo $d\ p[j]$ começa em zero e nunca exigimos encher a sacola exatamente, $d\ p[W]$ já é a resposta: “melhor que se consegue com peso **até** W ”.

Sobre o tipo do vetor: a resposta chega a $1000 \times 10^6 = 10^9$, que ainda cabe num inteiro de 32 bits com sinal (cujo máximo é 2147483647), mas com margem de menos de $2,2 \times 10^9$ — perto demais da borda para dormir tranquilo. Use inteiro de **64 bits** (long long em C/C++, long em Java, Long em Kotlin, i64 em Rust) no vetor da programação dinâmica e a dúvida desaparece.

Passos

1. Ler n e W e, em seguida, os n pares (w_i, v_i) para dois vetores.
2. Criar um vetor $d p$ de $W+1$ inteiros de 64 bits, todo zerado.
3. Para cada brinde i , com peso $w = w_i$ e valor $v = v_i$:
 1. para j indo de W **decrecendo** até w :
 1. calcular $\text{cand} \leftarrow d p[j - w] + v$;
 2. se $\text{cand} > d p[j]$, então $d p[j] \leftarrow \text{cand}$.
4. Imprimir $d p[W]$.

Complexidade

O laço externo roda n vezes e o interno roda no máximo $W+1$ vezes, com trabalho constante por iteração (uma soma, uma comparação e, às vezes, uma escrita). O tempo é $O(n \cdot W)$: no pior caso, $1000 \times 10^4 = 10^7$ operações — confortável para o tempo-limite mesmo nas linguagens interpretadas, porque cada operação é aritmética inteira sobre um vetor contíguo, sem alocação e sem estrutura de dados sofisticada.

A memória é $O(W)$: um vetor de 10001 inteiros de 64 bits, ou cerca de 80 KB — mais os $2n \leq 2000$ inteiros da entrada. A tabela bidimensional ingênua gastaria 1000×10001 inteiros de 64 bits, cerca de 80 MB; ela até caberia no limite de memória, mas o vetor unidimensional é mais simples, mais rápido (cabe inteiro na cache) e não corre risco nenhum.